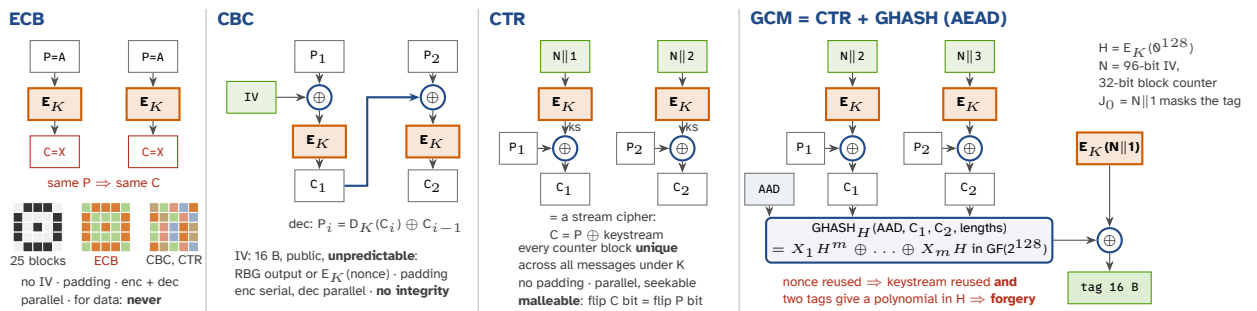


In one line: **AES** is a keyed *permutation* of one 16-byte block; a **mode** makes it a message cipher. **ECB** shows equal blocks, **CBC** chains (unpredictable IV + padding), **CTR** is a stream cipher (unique counters), **GCM** = CTR + a $\text{GF}(2^{128})$ MAC = **AEAD**. No confidentiality-only mode protects integrity; a repeated or predictable IV breaks them.



How it works

- AES** (FIPS 197, 2001; the 2023 update is editorial only): 128-bit block; 128/192/256-bit key → 10/12/14 rounds of SubBytes · ShiftRows · MixColumns · AddRoundKey (no MixColumns in the last). Same key + same block → same output, always — the rest is public. NIST proposed (Dec 2024) adding **Rijndael-256**: 256-bit block, one 256-bit key, for bulk data.
- Padding** (ECB, CBC): PKCS#7 (RFC 5652) appends n bytes of value n , 1–16 — a whole block of 0x10 if already aligned. Ciphertext stealing (CBC-CS1/2/3, SP 800-38A addendum, 2010) needs none; nor do CTR, GCM, ChaCha20: $|C| = |P|$.
- IV vs nonce** (SP 800-38A App. C): a CBC/CFB IV must be *unpredictable* — RBG output, or $E_K(\text{nonce})$; an OFB IV and the CTR counter blocks need only be *unique*. Both are public. TLS 1.0 used the last ciphertext block as the next IV: BEAST (2011); TLS 1.1 (2006) had already switched to explicit IVs.
- Birthday bound**: after $\sim 2^{b/2}$ blocks, ciphertext collisions leak plaintext XORs. 64-bit blocks (3DES, Blowfish): 2^{32} blocks = 32 GB; Sweet32 (2016) took a two-block HTTPS cookie with 785 GB. NIST then capped 3DES at 2^{20} blocks per key (2017); TDEA encryption is disallowed since 2024. AES: rekey well before 2^{64} blocks.

The SP 800-38A modes

mode	IV	parallel enc/dec	1 bit flipped in C_j	IV repeated
ECB	none	yes/yes	P_j random	equal blocks always show
CBC	unpredictable	no/yes	P_j random, same bit in P_j+1	equal prefixes show
CFB	unpredictable	no/yes	same bit in P_j , next b/s segments random	first segments' XOR leaks
OFB	unique	no/no (pre-compute)	same bit in P_j	two-time pad
CTR	unique counters	yes/yes	same bit in P_j	two-time pad

Random = each bit wrong with probability 1/2; same bit = exactly the flipped position — the attacker's steering wheel. **NIST, Oct 2026**: 38A is being revised (decided 2023) to limit ECB to uses other NIST rules allow — one-block jobs like IV generation, challenge-response, QUIC header protection; the SP 800-131A Rev. 3 *draft* 2024; Rev. 2 (still in force) disallows ECB for encryption. CBC...CTR stay acceptable, the CTS addendum folds into 38A, and NIST may deprecate these modes once a wide-block "accordion" (IR 8552) is approved.

AEAD constructions

construction	nonce	max message	if the nonce repeats
AES-GCM	96 bit, unique	2^{39} — 256 bits (≈ 64 GiB)	two-time pad + H recovered → forgeries
AES-CCM	7-13 B, unique	$< 2^{8L}$ B, $L = 15 - N $	two-time pad
ChaCha20-Poly1305	96 bit, unique	274 877 906 800 B (≈ 256 GiB)	two-time pad + one-time Poly1305 key reused
XChaCha20-Poly1305	192 bit, random OK	$\sim 2^{64}$ B	as ChaCha20 — collisions negligible
AES-GCM-SIV	96 bit	2^{36} B	leaks only <i>whether</i> two messages were equal
Ascon-AEAD128	128 bit, unique	2^{54} B per key	"shall be distinct" (SP 800-232 R3)

CCM (CBC-MAC, then CTR; two passes) runs BLE's link layer with a 4-byte MIC and 13-byte nonce, so $L = 2$: payloads < 64 KiB. ChaCha20 is $\sim 3\times$ AES on chips without AES instructions; libodium ships AES-GCM only for chips with them (x86 AES-NI, ARM Crypto extensions).

Malleability, worked (CTR, no MAC)

Known plaintext amount=100. XOR the ciphertext byte under 1 with "1" $\oplus$ "9" (= 0x08) — no key, no decryption — and the server reads amount=900. Encryption hid the value; it never protected it.

Example — the CBC padding oracle

Vaudenay (2002): "bad padding" answered unlike "bad MAC" (status, text, *timing*). $P_i = D_K(C_i) \oplus C_{i-1}$, so edit C_{i-1} : ≤ 256 tries a byte, no key.

```
def last_byte(prev, block):  
    # recover P[15] of block  
    for g in range(256):  
        fake = prev[:15] + bytes([g]) # we control C(i-1)  
        if oracle(fake + block) == "padding ok": # P'[15] is 0x01  
            d = g ^ 0x01 # d = D_K(block)[15]  
    return d ^ prev[15] # then force 0x02 0x02 for byte 14 ...
```

Then: TLS by timing (2003) · ASP.NET (MS10-070, 2010: web.config) · **Lucky 13** (2013): MAC-then-encrypt timing; 13 = 5 header + 8 sequence bytes in the MAC · **POODLE** (2014): SSL 3.0 padding is neither deterministic nor MAC-ed — a moved block passes 1 time in 256. Fix: **verify a MAC over IV + ciphertext before decrypting**, or use AEAD.

GCM, inside (SP 800-38D)

- $J_0 = \text{IV} \parallel 0^{31} \parallel 1$ for a 96-bit IV (others are GHASHed; NIST recommends 96 only); data from $\text{inc}_{32}(J_0)$; $T = \text{MSB}_t(E_K(J_0) \oplus \text{GHASH})$.
- $P(\text{IV repeats under a key}) \leq 2^{-32}$: deterministic IV = 32-bit **fixed field** (device) $\parallel$ 64-bit **invocation counter**, or random ≥ 96 bits and $\leq 2^{32}$ **messages per key**. TLS 1.3: nonce = static IV $\oplus$ record number. **Power loss must not repeat an IV**.
- Tags 128...96 bits; 64/32 only under App. C's caps. Targeted forgery $\approx n/2^t$ for n blocks, each success leaks H (Ferguson 2005). Rev. 1 (decided Mar 2024): no tags < 96 , TLS 1.3 IVs approved; pre-draft (Jun 2026): wGCM for Rijndael-256.
- Nonce reuse**: XOR of two tags = a known polynomial in H ; its roots give H (Joux, "forbidden attack", 2006) → tags for any ciphertext. 2016: 184 HTTPS servers repeated GCM nonces.

rekey before	limit per key
TLS 1.3 AES-GCM (RFC 8446)	$2^{4.5}$ full-size records
QUIC AES-GCM · AES-CCM (RFC 9001)	2^{23} · $2^{21.5}$ packets
QUIC ChaCha20-Poly1305	2^{36} failed forgeries (no confidentiality limit)
AES-GCM-SIV, random nonces	2^{32} msgs of 8 GiB, or 2^{48} of 32 MiB

Cipher + MAC: the order

Encrypt-then-MAC — $C, \text{MAC}(\text{IV} \parallel C)$ — rejects forgeries before decrypting: IPsec ESP, TLS RFC 7366 (2014), OpenSSH -etm MACs (6.2). **MAC-then-encrypt** (TLS ≤ 1.2 CBC) must decrypt first → Lucky 13, POODLE. **Encrypt-and-MAC** (SSH, RFC 4253: MAC over sequence number + plaintext): a MAC need not hide its input. Bellare-Namprempre (2000): only EtM is secure for every secure cipher + MAC pair. Separate keys. GCM, ChaCha20-Poly1305 MAC the ciphertext; TLS 1.3: AEAD only.

Beyond the basic modes

GCM-SIV (RFC 8452): the tag (POLYVAL) is the CTR IV, keys per nonce — misuse-resistant, two passes. **XTS** (SP 800-38E): XEX tweakable cipher + ciphertext stealing, tweak = data-unit number; no expansion, so **no tag**; storage only; data unit $\leq 2^{20}$ blocks; Rev. 1 draft (Sep 2026) cites IEEE 1619-2025. **Ascon-AEAD128** (SP 800-232, Aug 2025): no block cipher — a 320-bit permutation; 128-bit key, nonce, tag. **Not key-committing**: a GCM or ChaCha20-Poly1305 ciphertext can open under two keys — invisible salamanders (2018), partitioning oracles (2021, Shadowsocks).

Traps

- ! CCCrypt: default **CBC**, no IV → **all-zero IV**. Java SunJCE getInstance("AES") = **ECB**/PKCS5Padding (Google Play flags it). Go: no ECB, OFB/CFB deprecated (1.24).
- ! IV from the key, a constant IV, a counter reset on reinstall or shared by two devices; plaintext released before the tag check.

Remember

ECB leaks · CBC chains · CTR streams · GCM = CTR + tag · verify, then decrypt.